CablePro

Management System

Project Documentation

Complete Technical & User Reference Guide

Version 1.0.5

February 2026

Table of Contents

1. Introduction & Project Overview

1.1 Project Background

CablePro is a comprehensive web-based management system purpose-built for cable TV operators and MSOs (Multi-System Operators) operating in the Indian market. The system was developed to address the chronic inefficiencies that plague traditional paper-based and spreadsheet-driven billing workflows prevalent in the cable industry.

Before CablePro, cable operators typically managed hundreds of subscribers using physical ledgers, WhatsApp reminders, and standalone Excel sheets — workflows prone to data loss, human error, and missed collections. CablePro transforms this fragmented operation into a unified, mobile-friendly digital platform accessible from any device.

The application was designed with field operators in mind: someone who needs to quickly add a new subscriber, record a cash payment at the customer's doorstep, and check outstanding dues — all within seconds and without requiring any accounting expertise.

> **Market Context**
> India has over 100 million cable TV subscribers served by approximately 60,000 cable operators, the majority of whom are small to medium independent operators managing 200–2,000 subscribers without dedicated billing software.

1.2 Objectives & Goals

Primary Objectives

- Digitize the complete subscriber lifecycle — from onboarding to payment collection and status tracking.
- Provide real-time financial visibility through live dashboards and collection analytics.
- Reduce payment collection effort by surfacing overdue accounts and enabling one-tap payment recording.
- Eliminate data loss through cloud backup via Google Drive integration.
- Deliver a mobile-first experience optimized for operators who work primarily on smartphones.
- Support multiple plan durations (monthly, quarterly, semi-annual, annual) with automatic billing cycle tracking.

Secondary Objectives

- Enable data portability through CSV import/export for migration from legacy systems.
- Support dark mode for low-light usage environments.
- Provide PDF export for reports and collection statements that operators can share with franchise holders.

- Maintain minimal operational cost by using browser-local storage with optional cloud sync (no mandatory backend server cost).

Design Principles

Principle	Priority	Implementation
Mobile First	Critical	Bottom navigation bar, FAB buttons, touch-optimized targets, safe-area insets
Offline Capable	High	All data stored in localStorage; works without internet except for backup/auth
Zero Learning Curve	High	Familiar card-based UI; no training required for typical cable operators
Data Integrity	Critical	STB number uniqueness, phone validation, soft-delete with data preservation
Performance	Medium	Next.js static generation, optimized re-renders via useMemo/useCallback
Privacy	High	Per-user data isolation in localStorage; no cross-user data leakage

1.3 Key Features Summary

Customer Management

- Add, edit, and soft-delete customers with full profile information (name, phone, address, STB number, plan details).
- Search by name, phone number, or STB number in real-time.
- Filter customers by status: All, Active, Deactive, Overdue, Paid.
- Customer detail view with full payment history, outstanding balance, and quick contact actions (call/WhatsApp).
- Plan duration support: 1, 3, 6, and 12-month billing cycles with automatic coverage calculation.
- STB number uniqueness validation and 10-digit phone number format enforcement.

Payment Processing

- Record payments in seconds with pre-filled amount from customer's plan rate.
- Support for Cash and UPI/GPay/PhonePe payment methods.
- Duplicate payment prevention: blocks recording a payment if the customer's billing cycle is still active.
- Payment status management: Confirmed, Pending, and Cancelled states.
- Historical payment log per customer with date, method, billing period, and amount.

- Today / This Week / This Month summary statistics on the payments page.

Dashboard & Analytics

- Live business overview: total customers, active subscribers, outstanding balance, total paid this month.
- Interactive weekly collection bar chart built with Recharts.
- Payment method breakdown donut chart (Cash vs UPI).
- Recent activity feed with links to customer detail pages.
- Quick action shortcuts: Add Customer, Record Payment, Reports, Export PDF.

Reports & Export

- Month-level and custom date range filtering for financial reports.
- Collection trend area chart with daily data points.
- Payment method breakdown with percentage bars.
- Daily transaction breakdown table.
- Export to CSV (all fields) and PDF (formatted report with KPI summary).

Settings & Data Management

- Profile management with photo upload (max 500KB).
- CSV import for bulk customer onboarding from legacy systems.
- Google Drive cloud backup with configurable frequency (Manual, Every 3 Days, Weekly, Monthly).
- Automatic backup reminder banner after 3 days without backup.
- Soft account deletion with 7-day recovery window.

2. System Architecture

2.1 Technology Stack

CablePro is built as a modern Progressive Web Application (PWA) using a carefully selected stack that balances developer productivity, runtime performance, and deployment simplicity. Every technology choice was made with the constraint that the application must run entirely client-side with no mandatory backend server.

Layer	Technology	Version / Notes
Frontend Framework	Next.js (App Router)	v15 — React Server Components + Client Components
UI Library	React	v19 — with hooks-based state management
Language	TypeScript	v5 — strict mode enabled throughout
Styling	Tailwind CSS	v4 — JIT, custom design tokens via @theme
Charts	Recharts	v2 — BarChart, AreaChart, PieChart
Authentication	Firebase Auth	v10 — Email/Password + Google OAuth
Cloud Storage	Google Drive API	v3 REST API — file backup/restore
State Management	React Context + useState	Custom StoreProvider, no Redux/Zustand
Data Persistence	Browser localStorage	Per-user namespaced keys; ~5MB limit
Icons	Material Symbols (Google)	Outlined variant via CDN
Font	Inter (Google Fonts)	Variable font with subset optimization
Build Tool	Next.js / Turbopack	Production builds via Next.js CLI
Package Manager	npm	v10+
Deployment Platform	Vercel	Edge network, automatic HTTPS

2.2 Application Architecture

CablePro uses Next.js App Router with a clear separation between public routes (login, signup) and authenticated routes (dashboard, customers, payments, reports, settings). The architecture follows a layered pattern:

Directory Structure

```
src/
├── app/
│   ├── (app)/              # Authenticated route group
│   │   ├── layout.tsx      # Auth guard + AppShell wrapper
│   │   ├── dashboard/
│   │   │   └── page.tsx    # Dashboard with charts & quick actions
│   │   ├── customers/
│   │   │   ├── page.tsx    # Customer list with filters & CRUD
│   │   │   └── view/
│   │   │       ├── page.tsx
│   │   │       └── CustomerDetailClient.tsx
│   │   ├── payments/
│   │   │   ├── page.tsx    # Payment list & stats
│   │   │   └── new/
│   │   │       └── page.tsx # New payment form
│   │   ├── reports/
│   │   │   └── page.tsx    # Analytics & export
│   │   └── settings/
│   │       └── page.tsx    # Profile, backup, import/export
│   ├── page.tsx            # Login page (public)
│   ├── signup/
│   │   └── page.tsx        # Registration page (public)
│   ├── globals.css         # Tailwind + CSS custom properties
│   └── layout.tsx          # Root HTML layout + fonts
├── components/
│   ├── AppShell.tsx        # Sidebar + header + bottom nav
│   └── Modal.tsx           # Reusable dialog component
└── lib/
    ├── store.tsx           # Global state (StoreProvider + useStore)
    ├── types.ts            # TypeScript interfaces
    └── firebase.ts         # Firebase SDK initialization
```

Route Groups & Auth Guard

All authenticated pages live inside the (app) route group. The group's layout.tsx acts as a centralized auth guard — it checks for currentUser in the store and immediately redirects unauthenticated visitors to the login page. This ensures no protected content is ever rendered without authentication.

```
// src/app/(app)/layout.tsx — Auth Guard Pattern
export default function AppLayout({ children }) {
  const { currentUser } = useStore();
  const router = useRouter();

  useEffect(() => {
    if (!currentUser) router.replace("/");  // Redirect to login
  }, [currentUser, router]);

  if (!currentUser) return <Redirecting />;  // Show loading state

  return <AppShell>{children}</AppShell>;   // Render with shell
}
```

2.3 Data Flow & State Management

All application state lives in a single React Context (StoreContext) provided by StoreProvider at the root layout. This eliminates prop drilling and provides any component instant access to the full application state and all mutation functions via the useStore() hook.

State Persistence Strategy

Data is persisted to browser localStorage using user-scoped keys (cp_{userId}_{type}). This means every registered user has completely isolated data — switching accounts shows only that account's customers and payments.

Storage Key Pattern	Data Stored
cp_current_user	Currently logged-in user object (serialized JSON)
cp_{uid}_cust	Array of all Customer objects for this user
cp_{uid}_pay	Array of all Payment objects for this user
cp_{uid}_backup_freq	User's selected backup frequency setting
cp_{uid}_last_backup	Unix timestamp of last successful Drive backup
cp_deleted_users	Map of deleted user UIDs to deletion dates
cablepro_theme	Global theme preference ("light" or "dark")

State Update Flow

1. User performs an action (e.g., adds a customer via the modal form).
2. Component calls the appropriate store function (addCustomer()).
3. Store function validates the input data (uniqueness checks, required fields).
4. Store updates the in-memory React state via setState.
5. useEffect hooks watch the state changes and persist to localStorage.
6. All subscribed components re-render with the updated data automatically.

2.4 Authentication Architecture

CablePro supports three authentication methods, processed in priority order:

Authentication Methods

Method	Description & Use Case
Seed User (Demo)	Hard-coded operator user (username: operator, password: operator123). Loaded with pre-seeded customers and payments for demonstration purposes. Not created in Firebase — handled entirely in-memory.

Firebase Email/Password	Standard email & password authentication via Firebase Auth SDK. Supports account creation, login, and password reset via email link. Recommended for production operators.
Google OAuth (Sign In with Google)	One-tap Google sign-in via Firebase Auth. Also retrieves an OAuth access token scoped to drive.file for Google Drive backup functionality. Recommended for operators who want seamless cloud backup.

> *Important: Google OAuth login automatically retrieves the Drive access token, enabling cloud backup without a separate authorization step. Email/Password users must sign in with Google separately when initiating a backup.*

Soft Account Deletion

Account deletion in CablePro is "soft" — the user ID is recorded in a deletedUsers map with the deletion timestamp. The account cannot be logged into for 7 days (the recovery window), after which the data is effectively inaccessible. Before 7 days, the user can restore the account by clicking the recovery button on the login error message.

3. Installation & Setup Guide

3.1 Prerequisites

Before setting up CablePro locally, ensure your development machine meets the following requirements:

Requirement	Minimum Version	Notes
Node.js	v18.17+	LTS version recommended; required for Next.js 15
npm	v9+	Ships with Node.js; yarn and pnpm also supported
Git	v2.x	For cloning the repository
Modern Browser	Chrome 90+ / Firefox 88+ / Safari 15+	For development and testing
Firebase Account	Free Spark plan	Required for authentication setup
Google Account	Any	Required for Google OAuth and Drive backup testing
Code Editor	VS Code recommended	With ESLint and Tailwind CSS IntelliSense extensions

3.2 Development Environment Setup

Step 1: Clone the Repository

```
# Clone the repository
git clone https://github.com/your-org/cablepro.git
cd cablepro

# Verify structure
ls -la
# Should show: src/, public/, package.json, next.config.js, etc.
```

Step 2: Install Dependencies

```
# Install all npm dependencies
npm install

# This will install:
# - next, react, react-dom
# - typescript, @types/react, @types/node
# - tailwindcss, postcss, autoprefixer
# - firebase (Firebase SDK)
# - recharts (charts library)
# - docx (PDF generation)

# Verify installation
npm list --depth=0
```

Step 3: Start Development Server

```
# Start the Next.js dev server with Turbopack
npm run dev

# Server starts at: http://localhost:3000
# Hot reload enabled — changes reflect instantly

# Alternative: Standard webpack dev server
npm run dev -- --no-turbopack
```

3.3 Environment Configuration

CablePro's Firebase credentials are currently embedded in src/lib/firebase.ts. For production deployments, these should be moved to environment variables for security best practice.

Recommended: Environment Variables Setup

```
# Create .env.local in project root
NEXT_PUBLIC_FIREBASE_API_KEY=AIzaSyBzqWLoZmHoKg0zHBsb_HRFosdPYy3LcJ0
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=cablepro-58443.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=cablepro-58443
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=cablepro-58443.firebasestorage.app
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=710244325897
NEXT_PUBLIC_FIREBASE_APP_ID=1:710244325897:web:c90a2e24c29fa68558b6d8
NEXT_PUBLIC_FIREBASE_MEASUREMENT_ID=G-ZPGM34WXFP
```

Updated firebase.ts with Environment Variables

```
// src/lib/firebase.ts — Production-safe version
const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,
};
```

> *Never commit .env.local to version control. Add it to .gitignore. The NEXT_PUBLIC_ prefix exposes variables to the browser bundle, which is intentional for Firebase client-side SDK configuration.*

3.4 Firebase Configuration

Step 1: Create Firebase Project

7. Navigate to https://console.firebase.google.com/
8. Click "Add project" → name it (e.g., "cablepro-prod") → click Continue.
9. Optionally enable Google Analytics → Create Project.

Step 2: Enable Authentication Providers

10. In the Firebase Console, navigate to Build → Authentication → Sign-in method.
11. Enable Email/Password: Click → Enable → Save.
12. Enable Google: Click → Enable → enter your Project Support Email → Save.
13. Add your domain to Authorized Domains: go to Sign-in method → Authorized domains → Add domain.

Step 3: Register Web App

14. Go to Project Settings (gear icon) → Your apps → Add app → Web (</>).
15. Enter app nickname (e.g., "CablePro Web") → Register app.
16. Copy the firebaseConfig object shown — paste into src/lib/firebase.ts or .env.local.

Step 4: Enable Google Drive API (for Backup)

17. Go to https://console.cloud.google.com/ → select the same project.
18. Navigate to APIs & Services → Library → search "Google Drive API" → Enable.
19. Go to APIs & Services → OAuth consent screen → configure if needed.

3.5 Build & Deployment

Production Build

```
# Create optimized production build
npm run build

# Output:
# .next/                    — compiled Next.js bundle
# .next/static/             — hashed static assets
# Build summary shows route sizes and types

# Test production build locally
npm run start
# Serves production build at http://localhost:3000
```

Available Scripts

Script	Description
npm run dev	Start development server with hot reload (Turbopack)
npm run build	Create optimized production build
npm run start	Serve the production build locally
npm run lint	Run ESLint on all TypeScript/JSX files

4. User Interface & Navigation

4.1 Login Page

The login page (/) is the entry point for all users. It features a gradient background (indigo to white), CablePro branding, and three authentication pathways. The page is implemented in src/app/page.tsx with client-side form handling.

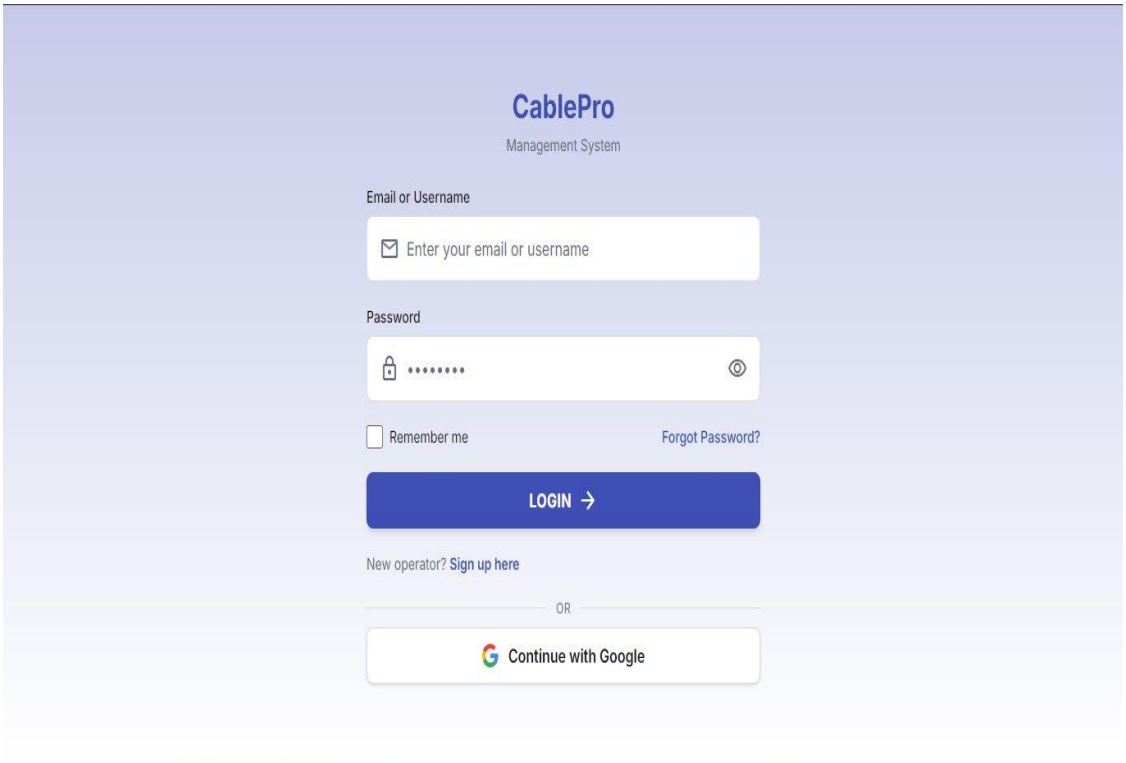

Figure 4.1 — CablePro Login Page with Email/Password and Google Sign-In options

Login Form Fields & Behavior

Field / Element	Behavior
Email or Username	Accepts both Firebase email addresses and the seed user's username ("operator"). Input type is text to support both formats.
Password	Masked by default with show/hide toggle (eye icon). Minimum validation only on submission.
Remember Me	Checkbox — currently cosmetic (Firebase manages session persistence by default).
Forgot Password?	Opens an inline modal for sending a Firebase password reset email to the entered address.
LOGIN button	Attempts seed user login first; falls back to Firebase email login. Shows a spinner while processing.
Sign up here	Navigates to /signup for new operator registration.

Continue with Google	Triggers Firebase Google OAuth popup flow; extracts Drive access token on success.

Authentication Priority Order

20. Check against SEED_USERS array (seed user login — works offline).
21. If not found, attempt Firebase Email/Password authentication.
22. If the Google button is clicked, trigger Google OAuth popup flow.

4.2 Application Shell & Navigation

After successful authentication, all pages are wrapped in AppShell (src/components/AppShell.tsx), which provides consistent navigation, theming, and layout structure.

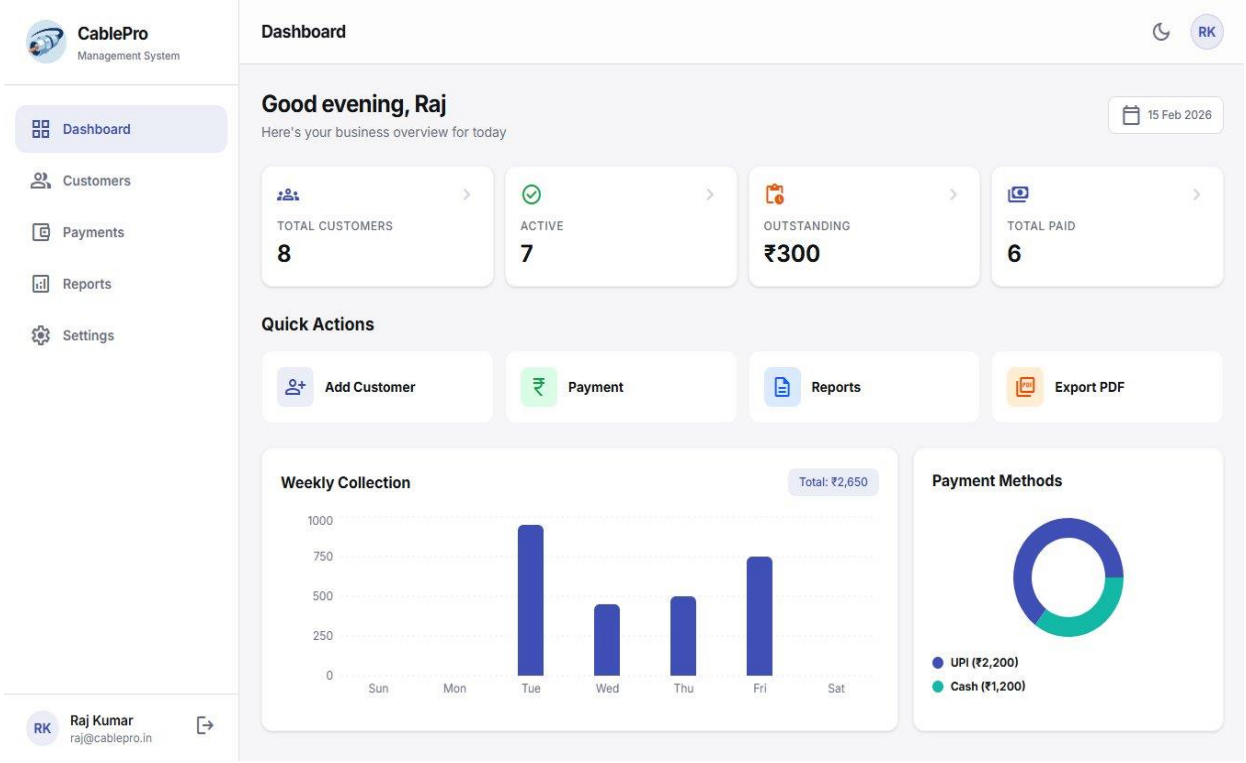

Figure 4.2 — Dashboard showing the Application Shell with sidebar navigation and top header

Layout Regions

Region	Description
Left Sidebar (Desktop)	Fixed-width 256px sidebar visible on lg+ screens. Contains logo, navigation links (Dashboard, Customers, Payments, Reports, Settings), and user profile with logout button. Active page is highlighted with brand blue background.

Top Header	Sticky header visible on all screen sizes. Shows hamburger menu (mobile only), current page title, dark mode toggle, and user avatar linking to Settings.
Bottom Navigation Bar (Mobile)	Fixed bottom bar on screens smaller than lg. Contains Home, Customers, central FAB (+), Reports, and Settings. Payments accessible via the FAB.
Main Content Area	Scrollable content region with responsive padding (16px mobile, 24px desktop). Bottom padding of 96px on mobile to clear the bottom nav.
Backup Reminder Banner	Conditional banner above main content reminding operators to back up if 3+ days since last backup.

Mobile Bottom Navigation

The bottom navigation implements a 4-item layout with a raised center FAB for the most frequent action (recording a payment). This pattern follows Material Design 3 conventions and is optimized for one-handed operation.

- Home (Dashboard) — grid_view icon
- Customers — group icon
- + FAB (center, raised) — links to /payments/new
- Reports — description icon
- Settings — settings icon

4.3 Dark Mode & Theming

CablePro supports a full dark mode toggled via the moon icon in the top header. The theme preference persists across sessions in localStorage (cablepro_theme key).

Theme Implementation

Dark mode is implemented using Tailwind's dark: variant and a class-based strategy. When dark mode is active, the class="dark" is added to the <html> element. The dark: variant in Tailwind v4 uses the @variant dark (&:where(.dark, .dark *)) pattern for accurate scoping.

```
// globals.css — Dark mode variant definition
@variant dark (&:where(.dark, .dark *));

// Custom design tokens for both modes:
@theme inline {
  --color-primary: #404fb5;           // Brand blue
  --color-bg-dark: #14151e;           // Page background (dark)
  --color-bg-card-dark: #1a1f26;      // Card background (dark)
  --color-border-dark: #2d3139;       // Borders (dark)
}
```

Custom Color System

Token	Light Mode Value	Usage
--color-primary	#404fb5	Brand blue — buttons, active states, headings
--color-teal	#0d9488	FAB buttons, WhatsApp actions, secondary CTAs
--color-success	#16a34a	Paid status, confirmed payments, call button
--color-orange	#ea580c	Outstanding dues, overdue alerts, Pay Now button
--color-danger	#dc2626	Delete actions, error states, cancellation
--color-bg-light	#f6f6f8	Page background (light mode)
--color-text-secondary	#6a6d81	Subtitles, helper text, inactive nav items

5. Module Documentation

5.1 Dashboard Module

The Dashboard (src/app/(app)/dashboard/page.tsx) is the primary landing page after login. It provides a comprehensive business overview with real-time KPI cards, interactive charts, quick action shortcuts, and a recent activity feed.

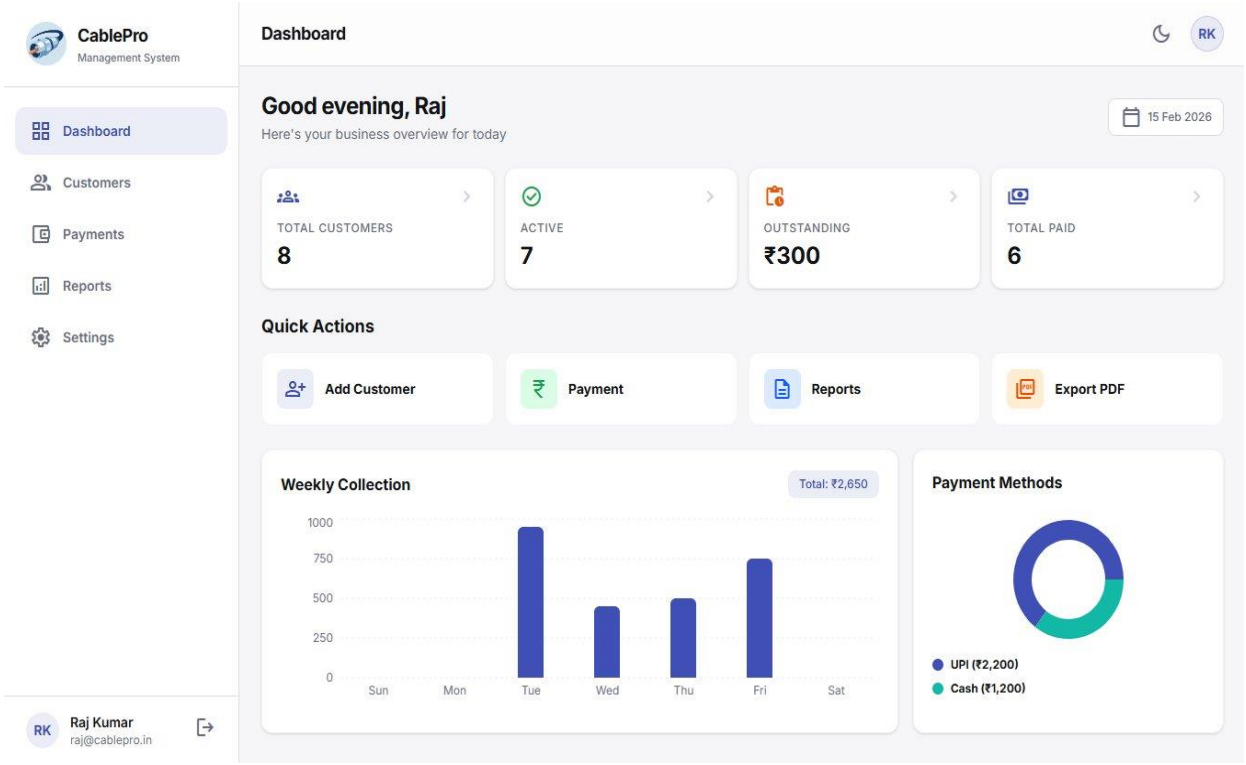

Figure 5.1 — Dashboard Module showing KPI cards, charts, and recent activity

KPI Summary Cards

Four summary cards display the most critical business metrics. Each card is clickable and navigates to the relevant filtered view in the Customers module.

KPI Card	Data Source	Navigation Target
Total Customers	customers.filter(c => !c.isDeleted).length	/customers?filter=All
Active	customers.filter(c => c.status === 'Active').length	/customers?filter=Active
Outstanding	Sum of monthlyRate for all overdue active customers	/customers?filter=Overdue
Total Paid	Count of distinct customer IDs with a confirmed payment this month	/customers?filter=Paid

Outstanding Balance Calculation

The outstanding balance calculation is one of the most critical pieces of business logic in CablePro. It accounts for multi-month plan durations, ensuring customers on quarterly or annual plans are not incorrectly flagged as overdue.

```
// Outstanding logic — checks if customer's plan is still active
const isCustomerCovered = (customer) => {
  if (customer.status !== "Active") return true; // Deactive = no dues

  const payments = getConfirmedPayments(customer.id)
                   .sort(byDateDesc);

  if (payments.length === 0) return false; // Never paid = overdue

  const lastPayDate = new Date(payments[0].paymentDate);
  const duration = customer.planDuration || 1; // months

  // Coverage ends (duration) months after last payment
  const coverageEnd = new Date(
    lastPayDate.getFullYear(),
    lastPayDate.getMonth() + duration,
    lastPayDate.getDate()
  );

  return new Date() < coverageEnd; // true = covered, false = overdue
};
```

> *Key insight: A customer on a 3-month plan who paid on January 1 is covered until April 1. Even if March arrives, they do not show as overdue. The next payment only becomes due on April 1.*

Quick Actions

Four quick action cards provide single-tap access to the most common operator workflows:

Action	Behavior
Add Customer	Navigates to /customers?action=add which triggers the customer add modal via URL param.
Payment	Navigates to /payments/new for immediate payment recording.
Reports	Navigates to /reports for financial analytics.
Export PDF	Generates and opens a print-ready HTML report for the current month's payments in a new tab.

Weekly Collection Bar Chart

The weekly collection chart shows payment amounts for the past 7 days aggregated by day of the week. Built with Recharts BarChart with the following configuration:

- Data: Sums confirmed payments grouped by day (Sun-Sat) for the rolling 7-day window.
- Bar color: #404fb5 (brand blue) with rounded top corners (radius [6,6,0,0]).

- Tooltip: Shows formatted INR amount on hover.
- Axes: No axis lines or tick lines — clean minimal presentation.
- Grid: Horizontal grid lines only (vertical=false) in light/dark adaptive colors.
- Total label: Displayed above the chart as total weekly collection.

Payment Methods Donut Chart

The payment methods chart shows the split between Cash and UPI payments as a donut chart (PieChart with innerRadius). The chart uses brand colors: #404fb5 for UPI and #14b8a6 (teal) for Cash.

> *Net Banking was intentionally removed from the payment methods. The codebase only supports Cash and UPI, reflecting the real-world usage pattern of Indian cable operators where UPI (GPay, PhonePe, Paytm) and cash dominate.*

Recent Activity Feed

The recent activity section shows the 5 most recent payment records (from the payments array, not filtered by date). Each item is clickable and navigates to the corresponding customer's detail page. Activity items show:

- Payment Received — green receipt icon — for Confirmed payments.
- Payment Cancelled — red cancel icon — for Cancelled payments.
- Customer name, amount, and payment method in the subtitle.
- Relative time (e.g., "2h ago", "3d ago") calculated from the payment's createdAt timestamp.

PDF Export Function

The "Export PDF" quick action generates a fully formatted HTML report and opens it in a new browser tab for printing. The report includes:

- CablePro branding header with the report month.
- Three KPI boxes: Total Collection (₹), Total Payments (count), Active Customers.
- Payment method breakdown (Cash vs UPI totals).
- Full transaction table sorted by date descending.
- Generated timestamp footer.

Implementation: The function builds an HTML string, opens a new window, writes the HTML, and calls window.print() after a 400ms delay to allow rendering. This approach requires no server-side PDF generation library.

5.2 Customer Management Module

The Customer Management module (src/app/(app)/customers/page.tsx) is the core operational hub of CablePro. It provides a searchable, filterable list of all subscribers with inline actions for every common operation.

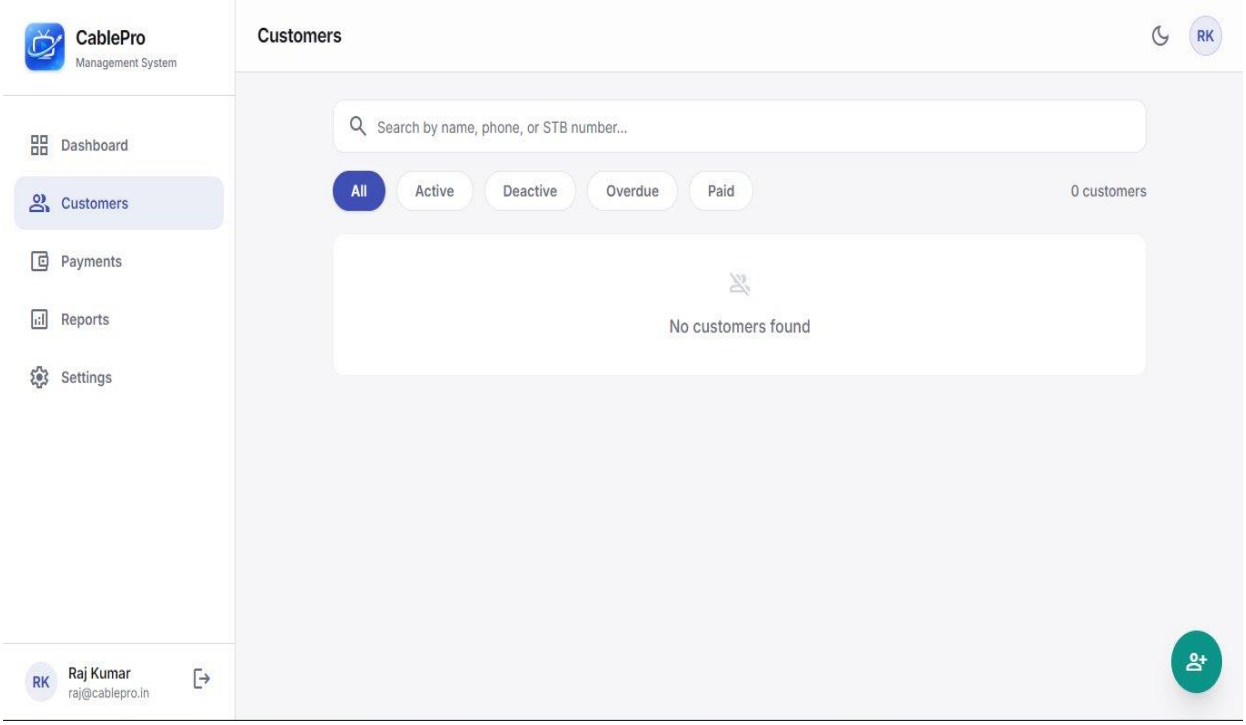

Figure 5.2 — Customers Module showing filter chips, search bar, and empty state

Customer List Features

Feature	Implementation Detail
Real-time Search	Filters by name (case-insensitive), phone number, or STB number as the user types. No debounce needed — localStorage queries are synchronous and fast.
Filter Chips	5 filters: All, Active, Deactive, Overdue, Paid. Overdue = active customers whose plan coverage has expired. Paid = active customers with no outstanding balance.
Customer Count	Displayed at the right end of the filter row — updates live as filters change.
Customer Cards	Each customer renders as a card showing: avatar (initials), name, status badge, phone, STB number, package name, outstanding balance, and action buttons.
Color-coded Avatars	Blue (primary) for paid/active customers; Orange for overdue; Gray for deactive.
Action Buttons	View (links to detail page), Call (tel: link), Edit (opens edit modal), Delete (opens confirmation dialog).
Pay Now Button	Orange button appears on cards with outstanding balance — links to /payments/new?customerId={id}.

Paid Badge	Green "Paid" badge appears on cards where outstanding = 0.
Deactive Badge	Gray "Deactive" badge on inactive customers — no payment button shown.

Add/Edit Customer Modal

The add and edit customer workflow uses the same modal form. Clicking the teal FAB (person_add icon, bottom-right) opens the Add Customer modal. Clicking Edit on a customer card pre-populates the form with existing data.

Form Fields

Field	Required	Validation Rules
Full Name	Yes	Non-empty string
Phone Number	Yes	Must be exactly 10 numeric digits (after stripping +91 and spaces). Must be unique across all customers.
Address	No	Free text; defaults to 'Unknown' if not provided on import
STB Number	Yes	Must be numeric only. Must be unique across all customers.
Plan Name	No	Free text (e.g., 'Ultra HD Gold Pack', 'Basic HD')
Plan Price (₹)	Yes	Numeric; defaults to 300 if not provided
Plan Duration	Yes	Dropdown: 1 Month, 3 Months, 6 Months, 12 Months
Status	Yes	Dropdown: Active or Deactive

Validation Logic

```
// Customer validation in store.tsx
const validateCustomer = (customer, idToExclude) => {
  // Required field checks
  if (!customer.name?.trim()) throw new Error("Customer Name is required.");
  if (!customer.phone?.trim()) throw new Error("Mobile Number is required.");

  // Phone: exactly 10 digits
  const cleanPhone = customer.phone.replace(/\s+/g, "").replace("+91", "");
  if (!/^\d{10}$/.test(cleanPhone))
    throw new Error("Mobile Number must be exactly 10 digits.");

  // Phone uniqueness (exclude self when editing)
  const duplicate = customers.find(c => {
    const existing = c.phone.replace(/\s+/g, "").replace("+91", "");
    return existing === cleanPhone && c.id !== idToExclude;
  });
  if (duplicate) throw new Error("Mobile Number already exists.");

  // STB: numeric only
  if (!customer.stbNumber?.trim()) throw new Error("STB Number is required.");
  if (!/^\d+$/.test(customer.stbNumber))
    throw new Error("STB Number must be numeric only.");
```

```
  // STB uniqueness
  if (customers.some(c => c.stbNumber === customer.stbNumber && c.id !== idToExclude))
    throw new Error("STB Number already exists.");
};
```

Soft Delete Implementation

Customer deletion in CablePro is non-destructive. Instead of removing the record, the deleteCustomer function marks the customer as deleted while preserving all associated payment history. This is critical for financial audit trails.

```
// Soft delete — preserves data, marks as deleted
const deleteCustomer = (id) => {
  setCustomers(prev => prev.map(c =>
    c.id === id ? {
      ...c,
      name: "Deleted Customer",    // Anonymize name
      phone: "",                   // Clear PII
      address: "",
      stbNumber: "",
      isDeleted: true,             // Hide from UI
      deletedAt: new Date().toISOString()
    } : c
  ));
  // Payment records retain the customer name as "Deleted Customer"
  setPayments(prev => prev.map(p =>
    p.customerId === id ? { ...p, customerName: "Deleted Customer" } : p
  ));
};
```

Customer Detail View

Clicking "View" on any customer card navigates to /customers/view?id={customerId}, which renders a detailed profile page (CustomerDetailClient.tsx). This page provides a comprehensive view of the customer's subscription and financial history.

Detail Page Sections

Section	Contents
Profile Header	Large avatar with online/offline status dot, full name, status badge with one-click toggle (Activate/Deactive), customer ID, Call button (tel: link), WhatsApp button (wa.me link).
Outstanding Balance	Color-coded balance card: orange border with amount for overdue, green border for clear balance. Shows last payment date.
Subscription Details	STB number, current plan name, plan price per month — each in a styled row with icon.
Personal Information	Mobile number, address, connection date — each in a styled row with icon.
Payment History	Full table of confirmed payments sorted by date descending: date, method, billing period (desktop only), amount.

Record Payment FAB	Teal floating button bottom-right linking to /payments/new?customerId={id} for quick payment recording.

URL Parameter Integration

The Customers module uses URL search parameters to enable deep-linking from other modules:

URL Parameter	Effect
/customers?action=add	Automatically opens the Add Customer modal when the page loads (triggered by Dashboard quick action).
/customers?filter=Overdue	Pre-selects the 'Overdue' filter chip (triggered by clicking the Outstanding KPI card on Dashboard).
/customers?filter=Active	Pre-selects the 'Active' filter chip (triggered by clicking the Active KPI card).
/customers?filter=Paid	Pre-selects the 'Paid' filter chip (triggered by clicking the Total Paid KPI card).
/customers/view?id={id}	Opens the Customer Detail page for the specified customer ID.

CSV Import Functionality

Operators can bulk-import customers from existing spreadsheets via the Settings page's Import CSV feature. The import function (importCustomers) is tolerant of various CSV formats:

Supported Column Headers (case-insensitive)

```
Supported header aliases:
  name / customer            → Customer full name
  phone / mobile             → Phone number
  address                    → Address
  stb number / stb           → STB/Set-Top Box number
  connection date            → Connection date (YYYY-MM-DD)
  package                    → Plan/package name
  monthly rate / plan price / amount → Monthly rate in ₹
  status                     → Active or Deactive
```

> *Import uses upsert logic — if a customer with the same ID already exists, it is skipped. New customers are appended to the existing list. Validation is not applied during import to allow bulk onboarding without interruption.*

5.3 Payment Processing Module

The Payment Processing module consists of two distinct pages: the Payments List (/payments) for viewing and managing existing payments, and the New Payment form (/payments/new) for recording a new payment.

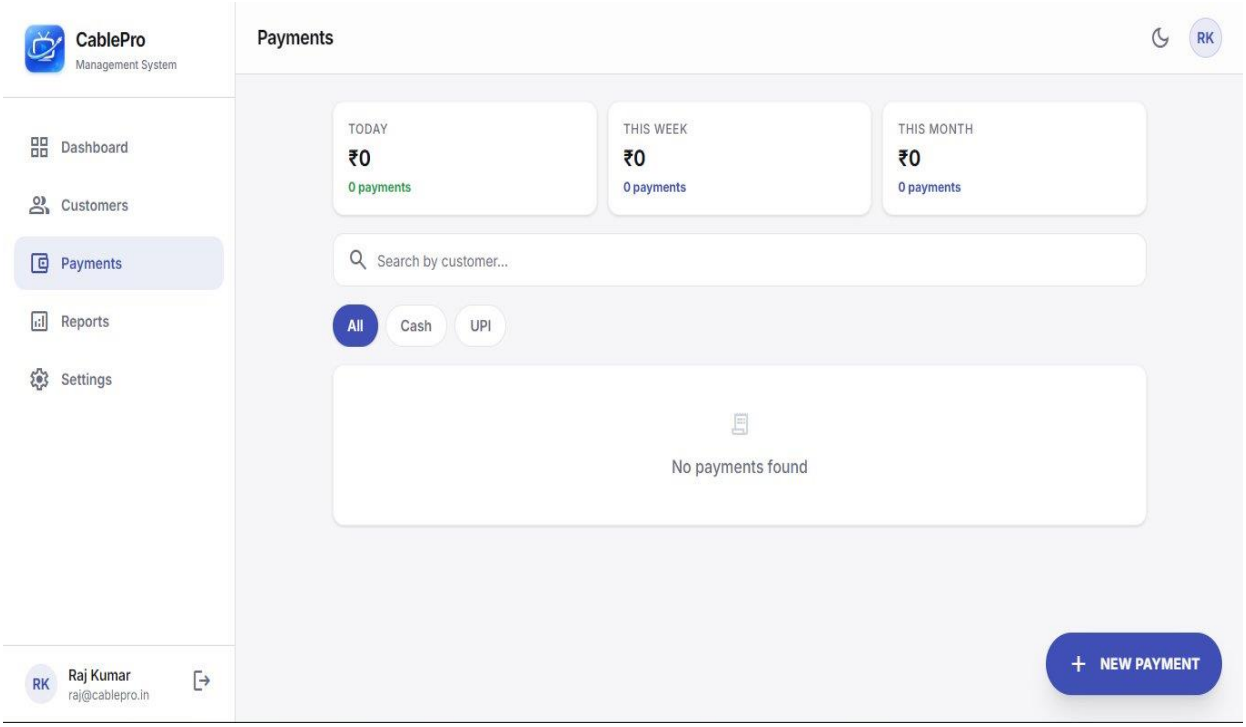

Figure 5.3 — Payments Module showing summary stats, search, filters, and empty state

Payments List Page

The payments list provides a tabular view of all payment transactions with real-time search, method filtering, and inline status management actions.

Summary Statistics Cards

Stat Card	Calculation
Today	Sum of confirmed payments where paymentDate === today's date (YYYY-MM-DD).
This Week	Sum of confirmed payments where paymentDate >= 7 days ago.
This Month	Sum of confirmed payments where paymentDate starts with current YYYY-MM.

Payment Table Columns

Column	Details
Customer	Customer display name from the payment record. Shows 'Deleted Customer' for removed accounts.

Date	Payment date (YYYY-MM-DD format). Hidden on mobile to save space.
Method	Cash or UPI. Hidden on mobile.
Amount	Payment amount in ₹, displayed in green (success color).
Status	Color-coded badge: green for Confirmed, amber for Pending, red for Cancelled.
Action	Context-sensitive buttons: Confirm/Cancel for Pending; Cancel (block icon) for Confirmed; dash for Cancelled.

Payment Status Transitions

Payments follow a simple state machine with three states:

```
Pending   →  Confirmed  (operator clicks the ✓ button)
Pending   →  Cancelled  (operator clicks the ✗ button)
Confirmed →  Cancelled  (operator clicks the block icon)

Note: Cancelled payments cannot be re-confirmed without creating a new payment.
Note: A customer's outstanding balance only considers Confirmed payments.
```

New Payment Form

The New Payment form (/payments/new) is accessible via the FAB on the Payments list, the + FAB in the bottom navigation, the 'Pay Now' button on customer cards, and the 'Record Payment' button on the customer detail page.

Customer Selection

The form begins with an intelligent customer search dropdown. Typing in the search field filters active customers by name, phone, or STB number. Selecting a customer auto-fills the amount field with their plan's monthly rate and displays their outstanding balance.

```
// Auto-fill behavior when customer is selected:
useEffect(() => {
  if (selectedCustomer) {
    setAmount(selectedCustomer.monthlyRate.toString());
    // Also calculates and displays outstanding balance
  }
}, [selectedCustomer]);
```

Duplicate Payment Prevention

Before saving, the system checks if the customer's current billing cycle is still active. If the customer paid for a 3-month plan in January, their coverage runs until April — recording another payment before April would be blocked.

```
// Duplicate payment guard
const custPayments = payments
  .filter(p => p.customerId === customer.id && p.status === "Confirmed")
  .sort(byBillingPeriodDesc);

if (custPayments.length > 0) {
```

```
  const duration = customer.planDuration || 1;

  // Parse "February 2026" → Date object for Feb 1, 2026
  const billingStart = new Date(lastBillingPeriod + " 1");

  // Coverage ends (duration) months after billing period start
  const coverageEnd = new Date(
    billingStart.getFullYear(),
    billingStart.getMonth() + duration,
    1
  );

  if (new Date() < coverageEnd) {
    // Show error: "Plan active until {coverageEnd}"
    // Block the payment submission
    return;
  }
}
```

> *This guard prevents operators from accidentally double-billing customers on multi-month plans. The error message tells the operator exactly when the next payment can be recorded.*

Payment Form Fields

Field	Type	Details
Select Customer	Searchable dropdown	Shows only Active, non-deleted customers. Displays plan duration badge and outstanding balance after selection.
Amount Paid	Number input	Pre-filled with customer's monthlyRate. Operator can override. Validated > 0.
Payment Method	Select dropdown	Options: Cash, UPI / GPay / PhonePe
Billing Period	Select dropdown	Shows current month + 11 past months in 'Month YYYY' format. Defaults to current month.
Notes	Textarea	Optional free-text notes about the payment.

Success State

On successful payment submission, the form transitions to a celebration state showing:

- Large green checkmark icon.
- 'Payment Recorded!' heading.
- Customer name and formatted amount in a card.
- 'Done' button (navigates to payments list).
- 'Add Another' button (resets form for the next payment).

Payment Method Filtering

The Payments list has three filter chips: All, Cash, and UPI. Selecting a method filter instantly shows only payments made by that method. Combined with the customer name search, operators can quickly find specific transactions.

Payment Cancellation

When an operator clicks the block/cancel icon on a Confirmed payment, the system calls updatePayment(id, { status: 'Cancelled' }). This immediately removes the payment from the 'Confirmed' pool, which recalculates the customer's outstanding balance — effectively marking them as overdue again.

Impact of Payment Cancellation

- The payment record remains visible in the payments list with a red 'Cancelled' badge.
- The customer's outstanding balance increases back to their monthly rate.
- The customer's status on the Customers page returns to orange (overdue).
- The Dashboard's Outstanding total increases accordingly.
- The Reports page excludes cancelled payments from collection totals.

Payment Data Integrity

The payment system enforces several integrity rules to maintain accurate financial records:

Rule	Enforcement
Customer must be Active	The new payment form only shows Active customers in the search dropdown. Deactive customers cannot receive payments.
Amount must be positive	Client-side validation rejects 0 or negative amounts before submission.
Customer must exist	getCustomer(id) is called before saving to verify the customer record still exists.
Billing cycle guard	The duplicate payment check prevents double-billing within an active coverage period.
Payment record immutability	Once a payment is Confirmed, the amount, date, method, and customer cannot be changed. Only the status can be updated.

5.4 Reports & Analytics Module

The Reports module (src/app/(app)/reports/page.tsx) provides comprehensive financial analytics with flexible date filtering, interactive charts, daily breakdowns, and export capabilities.

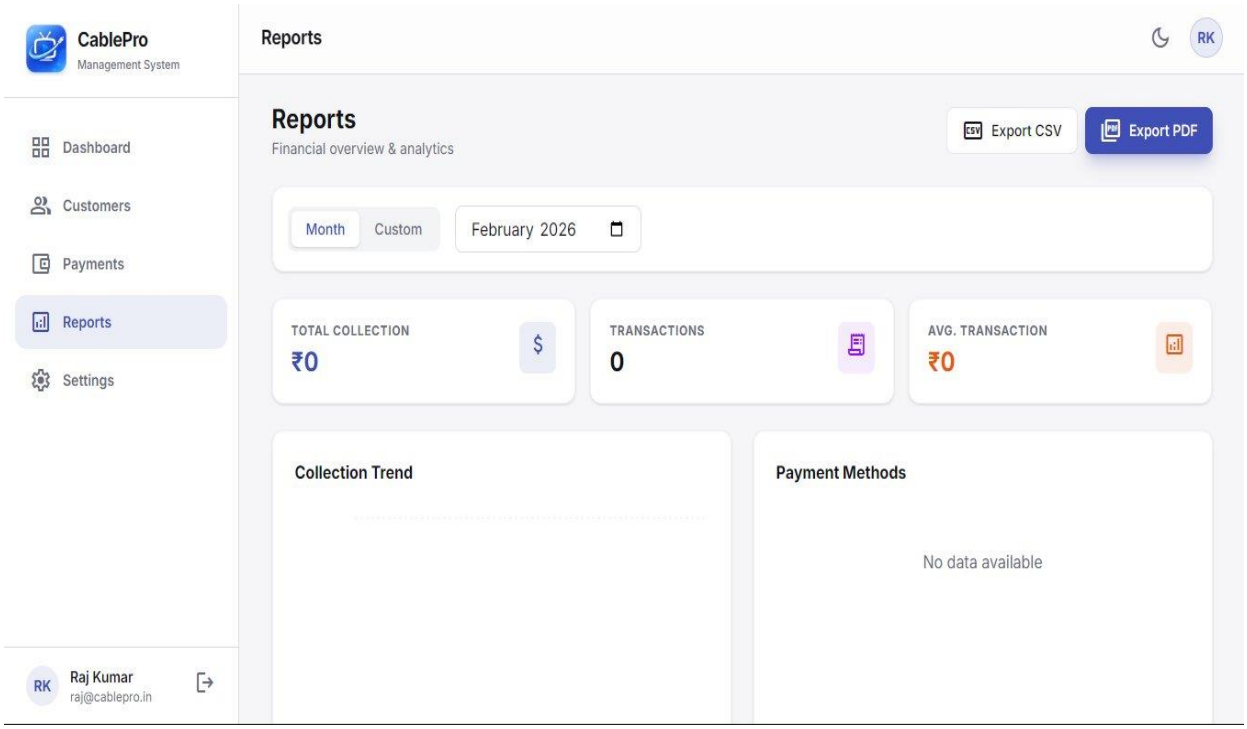

Figure 5.4 — Reports Module showing KPI summary, filter toolbar, and analytics sections

Date Range Filtering

The Reports module supports two filter modes accessible via toggle buttons:

Filter Mode	Behavior
Month	Shows a month picker input (type='month'). Defaults to the current month. Filters all data to payments within that calendar month.
Custom	Shows two date inputs (From and To). Allows filtering any arbitrary date range. Useful for weekly reports, quarterly reviews, or audits.

KPI Summary Cards

KPI Card	Formula	Color
Total Collection	Sum of amount for all filtered Confirmed payments	Brand Blue (₹)
Transactions	Count of filtered Confirmed payments	Gray (count)
Avg. Transaction	Total Collection ÷ Transactions (or 0 if no transactions)	Orange (₹)

Collection Trend Chart

The collection trend is an AreaChart that shows daily payment totals across the filtered period. The chart:

- Aggregates payments by date (paymentDate key).
- Fills gaps — all dates from first to last payment date are included, with 0 for days with no payments.
- Uses a gradient fill from #404fb5 (20% opacity) to transparent for visual depth.
- Area stroke is #404fb5 at 3px width.
- Tooltip shows the formatted ₹ amount for each day.
- Returns empty (no chart rendered) if there are no payments in the selected period.

```
// Trend data generation — fills date gaps
const trendData = useMemo(() => {
  const map = new Map(); // date → total amount
  filteredPayments.forEach(p => {
    map.set(p.paymentDate, (map.get(p.paymentDate) || 0) + p.amount);
  });

  const sortedDates = Array.from(map.keys()).sort();
  if (sortedDates.length === 0) return [];

  const data = [];
  const start = new Date(sortedDates[0]);
  const end = new Date(sortedDates[sortedDates.length - 1]);

  for (let d = start; d <= end; d.setDate(d.getDate() + 1)) {
    const iso = d.toISOString().slice(0, 10);
    data.push({
      name: `${d.getDate()} ${d.toLocaleString("en", { month: "short" })}`,
      amount: map.get(iso) || 0  // 0 for days with no payments
    });
  }
  return data;
}, [filteredPayments]);
```

Payment Methods Breakdown

The payment methods section shows a horizontal progress bar for each payment method with:

- Colored dot indicator (teal for Cash, blue for UPI).
- Method name and total amount (₹) for the period.
- Percentage bar showing the proportion relative to total collection.
- Percentage value displayed to the right of the bar.

Colors: Cash = #14b8a6 (teal), UPI = #404fb5 (brand blue). Any other method (legacy data) defaults to #94a3b8 (gray).

Daily Breakdown Table

Below the charts, a sortable table shows day-by-day collection data for the filtered period:

Column	Details
Date	Formatted as DD Mon YYYY (e.g., '15 Feb 2026') — sorted descending (most recent first).
Transactions	Count of confirmed payments on that date, displayed in a small blue badge.
Amount Collection	Total confirmed payment amount for that date, in green, right-aligned.

CSV Export

The 'Export CSV' button generates a CSV file containing all payments in the filtered date range. The file is downloaded automatically with a filename of report_{month}.csv (for month filter) or report_custom.csv (for custom range).

CSV Export Columns

```
CSV Headers:
Customer, Date, Method, Amount, Billing Period, Status

Example Row:
"Rajesh Kumar","2026-02-13","UPI",450,"February 2026","Confirmed"
```

PDF Export

The 'Export PDF' button generates a formatted HTML report and triggers the browser's print dialog. The exported PDF includes:

- CablePro branded header with the reporting period.
- KPI summary boxes: Total Collection, Transactions count.
- Payment method totals section.
- Complete transaction table with customer name, date, method, and amount.
- Page generation timestamp footer.

The implementation uses window.open() + window.print() pattern — no server-side PDF generation required. The operator can choose to Save as PDF in the print dialog.

Data Filtering Logic

```
// Core filter logic — applied to all report computations
const filteredPayments = useMemo(() => {
  return payments.filter(p => {
    if (p.status !== "Confirmed") return false; // Only confirmed

    if (filterType === "Month") {
      return p.paymentDate.startsWith(selectedMonth); // "2026-02"
    } else {
      // Custom range (inclusive)
      return p.paymentDate >= fromDate && p.paymentDate <= toDate;
    }
  });
```

```
}, [payments, filterType, selectedMonth, fromDate, toDate]);
```

Reports vs Dashboard Data

Metric	Key Difference
Dashboard 'This Month'	Uses thisMonthCollection from the store — computed at the store level, always reflects the current calendar month.
Reports 'Total Collection'	Computed within the Reports page based on the selected filter. Can reflect any month or date range, not just the current month.
Dashboard Charts	Weekly data (last 7 days rolling window) for the bar chart; all-time data for the pie chart.
Reports Charts	Daily granularity within the selected filter period (month or custom range).

5.5 Settings Module

The Settings module (src/app/(app)/settings/page.tsx) provides account management, data control, cloud backup, and danger zone operations.

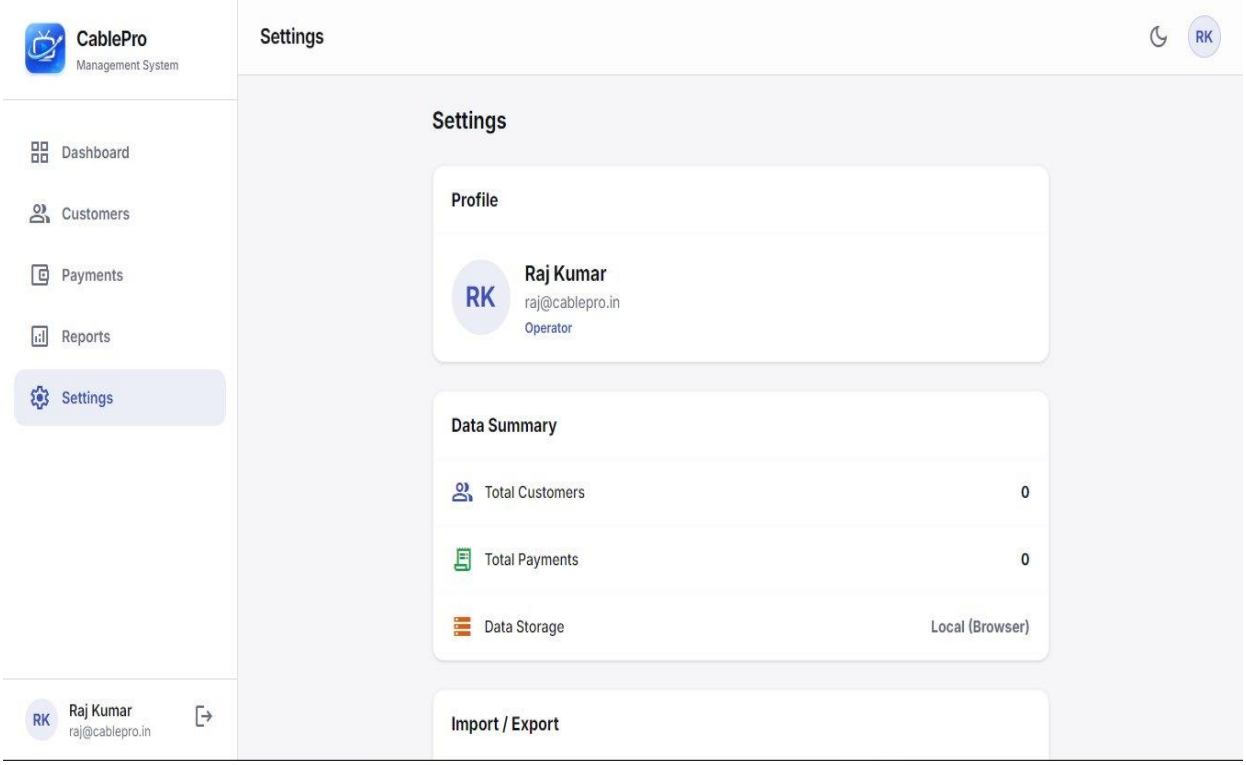

Figure 5.5 — Settings Module showing Profile, Data Summary, Import/Export, and Cloud Backup sections

Profile Section

The profile card shows the logged-in operator's information: avatar (initials or uploaded photo), full name, email address, and role (Operator). Clicking the avatar or the edit button opens the Edit Profile modal.

Edit Profile Modal

Field	Behavior
Profile Photo	Click the photo area or 'Change Photo' button to open the file picker. Images must be under 500KB. The selected image is read as a base64 data URL and stored in the user object (profileImage field).
Full Name	Text input pre-filled with current display name. Updated immediately on Save.

Data Summary Section

Three read-only counters show the scope of data stored for this account:

- Total Customers: Count of all customer records (including soft-deleted, for data integrity transparency).

- Total Payments: Count of all payment records in any status.
- Data Storage: Always shows 'Local (Browser)' — indicating localStorage is the active storage backend.

Import / Export Section

Export Customers CSV

Clicking 'Export Customers CSV' triggers the exportCustomersCSV store function, which generates a CSV file and initiates a browser download. The export includes all customers (including soft-deleted) with the following columns:

```
ID, Name, Phone, Address, STB Number, Package, Plan Price, Status, Connection Date

Use cases:
• Backup to external spreadsheet
• Share customer list with franchise holder
• Migrate to another system
```

Import Customers CSV

The Import CSV button opens a file picker. The selected CSV is parsed client-side (no server upload). Rows are mapped using header name matching and added to the existing customer list. Import behavior:

- Rows with empty name or phone are skipped (basic validation).
- Existing customer IDs are not duplicated (upsert by ID).
- All imported customers receive the current operator's ID as operatorId.
- Success/failure message is displayed inline after import completes.

> *Import does NOT enforce STB uniqueness or phone format validation. This is intentional to allow bulk migration without blocking on legacy data quality issues. Clean the data after import if needed.*

Cloud Backup (Google Drive)

The Cloud Backup section enables operators to save a complete data snapshot to Google Drive and restore from it at any time.

Backup Frequency Options

Option	Behavior
Manual	No automatic backup. Operator must click 'Backup Now' manually.
3 Days (labeled as 'daily')	System checks on app load and backs up automatically if 3+ days have passed since last backup.
Weekly	Automatic backup if 7+ days have passed since last backup.
Monthly	Automatic backup if 30+ days have passed since last backup.

Backup File Structure

The backup creates a JSON file named cablepro_backup.json inside a Google Drive folder named 'Cable Pro Backup'. The file contains:

```
{
  "customers": [...],     // All Customer objects
  "payments": [...],      // All Payment objects
  "deletedUsers": {...},  // Soft-deleted user records
  "settings": {
    "currentUser": {...}, // Operator profile
    "backupFrequency": "weekly"
  }
}
```

Backup Process Flow

23. Operator clicks 'Backup Now'.
24. System checks for a valid Google OAuth token (googleAccessToken).
25. If no token, triggers a Google Sign-In popup to obtain a drive.file scoped token.
26. Searches Google Drive for the 'Cable Pro Backup' folder — creates it if not found.
27. Searches the folder for existing cablepro_backup.json — uses PATCH (update) if found, POST (create) if not.
28. Uploads the serialized JSON data.
29. Updates lastBackupTime and dismisses the backup reminder banner.

Restore Process Flow

30. Operator clicks 'Restore'.
31. Confirmation dialog warns that current data will be overwritten.
32. System obtains Google OAuth token (same flow as backup).
33. Searches for 'Cable Pro Backup' folder and cablepro_backup.json inside it.
34. Downloads and parses the JSON file.
35. Replaces in-memory state: customers, payments, deletedUsers, and operator profile.
36. State changes trigger localStorage persistence via useEffect.

General Settings

Two display-only settings are shown in the General section:

- Language: English — fixed (localization not yet implemented).
- Currency: INR (₹) — fixed (INR is the only supported currency).

These items are styled as interactive rows with a chevron icon to suggest future editability.

Danger Zone

Sign Out

Clicking Sign Out shows a confirmation modal. On confirmation, the logout() function is called which: fires Firebase signOut(), clears the currentUser state, clears googleAccessToken, and redirects to the login page.

Delete Account

Account deletion is soft: the user's ID and deletion timestamp are recorded in the deletedUsers map. The operator is logged out immediately. Within 7 days, the account can be restored by logging in — a 'Restore Account' button appears on the login error message. After 7 days, the account is effectively permanently deleted.

Backup Reminder Banner

If the operator has configured a non-manual backup frequency AND 3+ days have passed since the last backup, a blue reminder banner appears below the top header across all pages (implemented in AppShell). The banner includes:

- A 'Backup Now' link that navigates to Settings.
- A dismiss (X) button that hides the banner for the current session.

6. Data Models & Type Definitions

CablePro's data layer is defined in src/lib/types.ts. Three core interfaces model the application's domain objects: Customer, Payment, and User.

6.1 Customer Data Model

```
// src/lib/types.ts
export interface Customer {
  id: string;           // Random 8-char alphanumeric ID (generated by genId())
  name: string;         // Full display name (e.g., "Rajesh Kumar")
  phone: string;        // Phone with +91 prefix (e.g., "+91 98765 43210")
  address: string;      // Free-text address
  stbNumber: string;    // Set-Top Box serial (numeric string, must be unique)
  connectionDate: string;  // ISO date: YYYY-MM-DD (when subscriber was added)
  packageName: string;  // Plan display name (e.g., "Ultra HD Gold Pack")
  monthlyRate: number;  // Plan price in ₹ (default: 300)
  planDuration: number; // Coverage duration in months: 1 | 3 | 6 | 12
  status: "Active" | "Deactive"; // Subscription status
  operatorId: string;   // ID of the operator who manages this customer
  createdAt: string;    // ISO date: YYYY-MM-DD (record creation date)
  isDeleted?: boolean;  // Soft delete flag (undefined = not deleted)
  deletedAt?: string;   // ISO timestamp when deleted (optional)
}
```

Customer Field Notes

Field	Important Notes
id	Generated by genId() — Math.random().toString(36).substring(2,10). Not cryptographically secure but sufficient for localStorage-based isolation.
phone	Stored with +91 prefix and spaces as entered. Stripped to 10 digits for validation and uniqueness comparison.
stbNumber	Must be purely numeric. Validated on add and edit. Used as a primary subscriber identifier in the Indian cable industry.
planDuration	Controls the billing cycle length. A customer with planDuration=3 who paid in January is covered through March.
monthlyRate	Represents the per-billing-cycle rate, not a true monthly rate. A 3-month plan at ₹450 = ₹450 per quarter (not ₹450/month × 3).
isDeleted	When true, the customer is hidden from the Customers list and search results, but their payment history is preserved.

6.2 Payment Data Model

```
export interface Payment {
  id: string;              // Random 8-char alphanumeric ID
  customerId: string;      // FK → Customer.id
  customerName: string;    // Denormalized name (survives customer deletion)
  amount: number;          // Payment amount in ₹
  paymentDate: string;     // ISO date: YYYY-MM-DD (when payment was made)
  paymentMethod: "Cash" | "UPI"; // Only Cash and UPI supported
```

```
  billingPeriod: string;   // Human-readable: "February 2026"
  status: "Confirmed" | "Pending" | "Cancelled";
  notes: string;           // Optional free-text notes
  createdAt: string;       // ISO timestamp: full datetime of record creation
}
```

Payment Field Notes

Field	Important Notes
customerName	Denormalized copy of the customer's name at the time of payment. This ensures payment records remain meaningful even after a customer is soft-deleted (where the Customer.name becomes 'Deleted Customer').
paymentDate	The actual date of payment (YYYY-MM-DD). Different from createdAt — an operator might enter a payment made yesterday. Used for all date-based filtering and reporting.
billingPeriod	Human-readable month string (e.g., 'February 2026'). Used for the duplicate payment guard and displayed in the payment history table. Stored as 'Month YYYY' to be both readable and parseable.
status	Controls whether a payment counts toward outstanding balance calculation. Only 'Confirmed' payments reduce a customer's outstanding balance. Pending and Cancelled payments are ignored in all financial computations.
createdAt	Full ISO timestamp including time. Used to calculate relative time for the 'Recent Activity' feed on Dashboard.

6.3 User Data Model

```
export interface User {
  id: string;          // Firebase UID (for Firebase users) or "u2" (seed user)
  username: string;    // For seed user only; email prefix for Firebase users
  password: string;    // Stored for seed user only; always "" for Firebase users
  fullName: string;    // Display name (from Firebase displayName or signup form)
  role: "admin" | "operator"; // Currently all users are "operator"
  email: string;       // Email address (from Firebase or signup form)
  profileImage?: string; // Base64 data URL of profile photo (optional)
  isDeleted?: boolean; // Not used on User; deletion tracked in deletedUsers map
  deletedAt?: string;  // Not used on User (see deletedUsers in store)
}
```

> *Security note: The password field is only non-empty for the hard-coded SEED_USERS array. Firebase-authenticated users never have their password stored in the User object — Firebase handles credential management.*

7. Store & State Management

7.1 Store Architecture

CablePro uses a custom React Context-based store (src/lib/store.tsx) instead of a third-party state management library. The store is a single StoreProvider component that wraps the entire application and exposes its API through the useStore() hook.

Why Context Over Redux/Zustand?

- The application's state shape is simple and well-defined — a list of customers and payments per user.
- No complex state derivations requiring middleware (like Redux Thunk/Saga).
- Single source of truth with simple update patterns suits useContext + useState perfectly.
- Eliminates a dependency and keeps the bundle smaller.

Store Interface (AppStore type)

The store exposes the following categories of values and functions:

Category	Members
Authentication	currentUser, login, signup, firebaseEmailSignup, firebaseEmailLogin, googleLogin, logout, deleteAccount, restoreAccount, sendPasswordReset, updateUserProfile
Customer CRUD	customers, addCustomer, updateCustomer, deleteCustomer, getCustomer, importCustomers
Payment CRUD	payments, addPayment, updatePayment, getPaymentsForCustomer
Computed Values	totalCustomers, activeCustomers, totalPaidCustomers, totalOutstanding, todayCollection, thisMonthCollection, recentActivity
Export	exportCustomersCSV, exportPaymentsCSV
Cloud Backup	backupToDrive, restoreFromDrive, backupFrequency, setBackupFrequency
UI State	theme, toggleTheme, showBackupReminder, setShowBackupReminder
Other	hydrated, googleAccessToken

Hydration Pattern

The store uses a hydrated flag to prevent SSR/client mismatch. On initial mount, the store loads from localStorage and sets hydrated = true. Components that depend on localStorage data should check hydrated before rendering content that could cause a flash.

```
// Hydration check in CustomerDetailClient.tsx
```

```
if (!hydrated) {
  return <div>Loading customer data...</div>;
}
```

7.2 Customer Operations

addCustomer

```
const addCustomer = (data: Omit<Customer, "id" | "createdAt">) => {
  validateCustomer(data);  // Throws on validation failure
  const customer = { ...data, id: genId(), createdAt: today() };
  setCustomers(prev => [customer, ...prev]);  // Prepend (newest first)
  return customer;
};
```

updateCustomer

```
const updateCustomer = (id: string, updates: Partial<Customer>) => {
  const existing = customers.find(c => c.id === id);
  if (existing) {
    validateCustomer({ ...existing, ...updates }, id); // Pass id to exclude self
  }
  setCustomers(prev => prev.map(c => c.id === id ? { ...c, ...updates } : c));
};
```

deleteCustomer (Soft Delete)

```
const deleteCustomer = (id: string) => {
  setCustomers(prev => prev.map(c =>
    c.id === id ? {
      ...c,
      name: "Deleted Customer",
      phone: "", address: "", stbNumber: "",
      isDeleted: true,
      deletedAt: new Date().toISOString()
    } : c
  ));
  // Anonymize payment records
  setPayments(prev => prev.map(p =>
    p.customerId === id ? { ...p, customerName: "Deleted Customer" } : p
  ));
};
```

7.3 Payment Operations

addPayment

```
const addPayment = (data: Omit<Payment, "id" | "createdAt">) => {
  const payment = { ...data, id: genId(), createdAt: new Date().toISOString() };
  setPayments(prev => [payment, ...prev]);  // Prepend (newest first)
  return payment;
};
```

updatePayment

```
const updatePayment = (id: string, updates: Partial<Payment>) => {
  setPayments(prev => prev.map(p => p.id === id ? { ...p, ...updates } : p));
};
// Primary use: status updates (Confirmed/Cancelled)
// Example: updatePayment("p1", { status: "Confirmed" });
```

7.4 Computed Properties

Several derived values are computed in the store on every render to ensure they always reflect the current state. These use simple array operations (no memoization at the store level) since the arrays are typically small (< 2000 entries).

```
const totalCustomers = customers.filter(c => !c.isDeleted).length;

const activeCustomers = customers.filter(c =>
  c.status === "Active" && !c.isDeleted
).length;

const totalPaidCustomers = Array.from(paidThisMonth)
  .filter(cid => nonDeletedIds.has(cid)).length;

const totalOutstanding = customers
  .filter(c => c.status === "Active" && !c.isDeleted && !isCustomerCovered(c))
  .reduce((sum, c) => sum + c.monthlyRate, 0);

const todayCollection = payments
  .filter(p => p.paymentDate === today && p.status === "Confirmed")
  .reduce((sum, p) => sum + p.amount, 0);

const thisMonthCollection = payments
  .filter(p => p.paymentDate.startsWith(currentMonth) && p.status === "Confirmed")
  .reduce((sum, p) => sum + p.amount, 0);
```

8. API & Firebase Integration

8.1 Firebase Authentication

CablePro uses Firebase Authentication SDK v10 (modular API). The firebase.ts file initializes the Firebase app and exports the required auth functions.

```
// src/lib/firebase.ts — exported functions
import {
  getAuth,
  GoogleAuthProvider,
  signInWithPopup,
  signOut,
  onAuthStateChanged,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  sendPasswordResetEmail,
  updateProfile,
} from "firebase/auth";

// Key configurations:
const googleProvider = new GoogleAuthProvider();
googleProvider.addScope("https://www.googleapis.com/auth/drive.file");
// ↑ Requests Drive access during Google Sign-In
```

Email/Password Signup (firebaseEmailSignup)

```
const firebaseEmailSignup = async ({ fullName, email, password }) => {
  try {
    const cred = await createUserWithEmailAndPassword(auth, email, password);
    // Set display name immediately after account creation
    await updateProfile(cred.user, { displayName: fullName });
    setCurrentUser(toLocal(cred.user));
    return { ok: true };
  } catch (err) {
    // Map Firebase error codes to user-friendly messages
    if (err.message.includes("email-already-in-use"))
      return { ok: false, error: "This email is already registered." };
    if (err.message.includes("weak-password"))
      return { ok: false, error: "Password must be at least 6 characters." };
    return { ok: false, error: err.message };
  }
};
```

Email/Password Login (firebaseEmailLogin)

```
const firebaseEmailLogin = async (email, password) => {
  try {
    const cred = await signInWithEmailAndPassword(auth, email, password);

    // Check for soft-deleted account
    const check = checkDeleted(cred.user.uid);
    if (check.isDeleted) {
      await firebaseSignOut(auth); // Sign out immediately
      return { ok: false, isDeleted: true, uid: cred.user.uid };
    }

    setCurrentUser(toLocal(cred.user));
```

```
    return { ok: true };
  } catch (err) {
    if (err.message.includes("invalid-credential"))
      return { ok: false, error: "Invalid email or password." };
    return { ok: false, error: err.message };
  }
};
```

Google OAuth Login (googleLogin)

```
const googleLogin = async () => {
  try {
    const provider = new GoogleAuthProvider();
    provider.addScope("https://www.googleapis.com/auth/drive.file");

    const result = await signInWithPopup(auth, provider);

    // Extract and cache the Google OAuth access token
    const credential = GoogleAuthProvider.credentialFromResult(result);
    const token = credential?.accessToken;
    if (token) setGoogleAccessToken(token); // Used for Drive backup

    const check = checkDeleted(result.user.uid);
    if (check.isDeleted) {
      await firebaseSignOut(auth);
      return { ok: false, isDeleted: true, uid: result.user.uid };
    }

    setCurrentUser(toLocal(result.user));
    return { ok: true };
  } catch (err) {
    if (err.message.includes("popup-closed-by-user"))
      return { ok: false, error: "Sign-in popup was closed." };
    return { ok: false, error: err.message };
  }
};
```

Password Reset (sendPasswordReset)

```
const sendPasswordReset = async (email) => {
  try {
    await sendPasswordResetEmail(auth, email);
    return { ok: true };
  } catch (err) {
    if (err.message.includes("user-not-found"))
      return { ok: false, error: "No user found with this email." };
    return { ok: false, error: err.message };
  }
};
```

8.2 Google Drive Backup API

The backup and restore functionality uses the Google Drive REST API v3 directly from the browser using the OAuth access token obtained during Google Sign-In.

API Endpoints Used

Operation	Endpoint	Method
List folders	drive.googleapis.com/drive/v3/files?q=...	GET
Create folder	drive.googleapis.com/drive/v3/files	POST
List files in folder	drive.googleapis.com/drive/v3/files?q=...	GET
Upload new file	drive.googleapis.com/upload/drive/v3/files?uploadType=multipart	POST
Update existing file	drive.googleapis.com/upload/drive/v3/files/{id}?uploadType=media	PATCH
Download file	drive.googleapis.com/drive/v3/files/{id}?alt=media	GET

Backup File Upload Logic

The backup logic first checks if the backup file already exists (to update it) or creates it fresh:

```
// Simplified backup upload logic
if (existingFile) {
  // Update existing file (PATCH with media upload type)
  await fetch(

`https://www.googleapis.com/upload/drive/v3/files/${existingFile.id}?uploadType=media`,
    { method: "PATCH", headers: { Authorization: `Bearer ${token}` }, body: fileContent }
  );
} else {
  // Create new file with multipart (metadata + content)
  const form = new FormData();
  form.append('metadata', new Blob([JSON.stringify({ name: FILE_NAME, parents:
[folderId] })],
    { type: 'application/json' }));
  form.append('file', new Blob([fileContent], { type: 'application/json' }));

  await fetch(
    "https://www.googleapis.com/upload/drive/v3/files?uploadType=multipart",
    { method: "POST", headers: { Authorization: `Bearer ${token}` }, body: form }
  );
}
```

8.3 Authentication Flows

New Operator Registration Flow

37. Operator navigates to /signup.
38. Fills in: Full Name, Email, Password, Confirm Password.
39. System validates: all fields required, password >= 6 chars, passwords match.
40. firebaseEmailSignup() is called → creates Firebase account → sets displayName.
41. On success: operator is immediately logged out (to force login confirmation).
42. Redirect to / with ?signup=success query param.
43. Login page shows success banner: 'Account created. You can now log in.'

Session Persistence

Firebase Authentication SDK automatically persists the user session in IndexedDB (Firebase's default persistence). When the page is refreshed, Firebase silently restores the session. CablePro additionally stores the currentUser object in localStorage (cp_current_user) for immediate access without waiting for Firebase's async session check.

Token Expiry & Refresh

Google OAuth access tokens expire after 1 hour. When an access token has expired and the operator tries to backup, the googleLogin popup is triggered again to obtain a fresh token. This is handled in both backupToDrive and restoreFromDrive:

```
let token = googleAccessToken;
if (!token) {
  // Obtain fresh token via popup
  const result = await signInWithPopup(auth, provider);
  const credential = GoogleAuthProvider.credentialFromResult(result);
  token = credential?.accessToken || null;
  if (token) setGoogleAccessToken(token);
}
if (!token) return { ok: false, error: "Failed to get access token." };
```

9. Security & Data Privacy

9.1 Authentication Security

Security Concern	CablePro's Approach
Password storage	Passwords are NEVER stored for Firebase users. The User interface has a password field only for the hard-coded seed user (operator123) in development. Firebase manages all credential security.
Session management	Firebase SDK manages session tokens in IndexedDB. CablePro does not handle raw JWTs. Sessions persist until explicit logout.
Account takeover prevention	Firebase provides built-in brute-force protection, suspicious activity detection, and multi-factor authentication (not currently enabled in CablePro).
Soft delete recovery window	Deleted accounts cannot log in for 7 days. This prevents immediate account access after deletion, allowing recovery of accidentally deleted accounts.
Cross-user data isolation	All localStorage keys are prefixed with cp_{userId}_. Different users' data never mixes in the same browser unless they log into the same browser.

9.2 Data Storage Security

> *Important limitation: CablePro stores all operational data in browser localStorage. This means data is device-specific and accessible to any JavaScript code running on the same origin. For production use with sensitive subscriber data, operators are encouraged to use the Google Drive backup feature to maintain an encrypted copy.*

Risk	Mitigation
localStorage access by browser extensions	No mitigation beyond choosing a browser with a restricted extension policy.
Data loss on browser clear/reinstall	Google Drive backup provides cloud persistence. Operators should back up regularly.
Unauthorized access to shared device	No biometric/PIN lock on the web app. Operators should use device-level screen lock.
Firebase credentials exposure	Firebase API keys are public by design (they identify the project, not authorize access). Firebase Security Rules protect backend resources.

9.3 Input Validation

CablePro performs client-side validation for all user inputs. While client-side validation improves UX, it also serves as the only validation layer (no server to validate against).

Input	Validation Applied
Customer phone	Stripped of +91 and spaces; must be exactly 10 numeric digits; must be unique.
Customer STB	Must be numeric only (regex /^\d+$/); must be unique.
Customer name	Non-empty string; no format restriction.
Payment amount	Must be a positive number (> 0).
Password (signup)	Minimum 6 characters; must match confirmation.
Profile photo	Maximum 500KB file size; must be an image (accept='image/*').
CSV import	Rows with empty name or phone are skipped; no further format validation.

10. Testing & Quality Assurance

10.1 Manual Testing Checklist

Authentication

- [ ] Login with seed user (operator / operator123) → Dashboard loads with demo data.
- [ ] Login with incorrect password → Error message shown.
- [ ] Sign up with new email → Redirected to login with success banner.
- [ ] Login with newly created account → Empty customer list.
- [ ] Google Sign-In → Redirected to Dashboard.
- [ ] Forgot Password → Reset email sent (check inbox).
- [ ] Delete account → Cannot log in; 'Restore Account' button appears.
- [ ] Restore account → Login succeeds.

Customer Management

- [ ] Add customer with all required fields → Customer appears in list.
- [ ] Add duplicate STB number → Validation error shown.
- [ ] Add duplicate phone number → Validation error shown.
- [ ] Add phone with fewer than 10 digits → Validation error shown.
- [ ] Edit customer → Changes persist on refresh.
- [ ] Delete customer → Disappears from list; payment history preserved.
- [ ] Search by name → Filters results correctly.
- [ ] Search by STB → Filters results correctly.
- [ ] Filter 'Overdue' → Shows only customers with outstanding dues.
- [ ] Filter 'Paid' → Shows only customers with no outstanding balance.
- [ ] Click 'View' → Customer detail page loads with correct data.
- [ ] Click 'Call' → Device phone dialer opens.
- [ ] Click 'WhatsApp' → WhatsApp opens (mobile) or web.whatsapp.com (desktop).

Payment Processing

- [ ] New payment: select customer → Amount auto-fills with plan rate.
- [ ] Record payment for active customer → Appears in payments list.
- [ ] Record duplicate payment within billing cycle → Error message shown.
- [ ] Cancel a confirmed payment → Customer shows as overdue again.
- [ ] Confirm a pending payment → Customer shows as paid.
- [ ] Payment from 'Pay Now' button → Customer ID pre-selected.

Reports

- [ ] Month filter → Shows only payments for selected month.
- [ ] Custom date range → Shows only payments in range.
- [ ] Export CSV → Downloads file with correct data.
- [ ] Export PDF → Opens print dialog with formatted report.

Settings & Data

- [ ] Export Customers CSV → Downloads file.
- [ ] Import CSV → Customers appear in list.
- [ ] Dark mode toggle → Persists across page refresh.
- [ ] Google Drive Backup → File appears in Google Drive.
- [ ] Google Drive Restore → Data restored from Drive file.

10.2 Browser Compatibility

Browser	Support Level	Notes
Chrome 90+	Full Support	Primary development & testing target
Firefox 88+	Full Support	Tested for all core workflows
Safari 15+	Full Support	Some CSS differences; safe-area-inset for notch devices
Edge 90+	Full Support	Chromium-based; same as Chrome
Chrome Mobile	Full Support	Primary mobile target; bottom nav optimized for touch
Safari iOS 15+	Full Support	PWA installation supported; notch handling via env()
Samsung Internet 14+	Partial Support	Core features work; some CSS animations may differ
IE11	Not Supported	Not targeted; uses ES modules and modern CSS

10.3 Performance Considerations

Concern	Current Approach & Limits
Data volume limit	localStorage has a ~5MB limit per origin. With ~1KB per customer and ~0.5KB per payment, the system can hold ~3,000 customers and ~5,000 payments before approaching limits.
Render performance	Customer list renders all customers in a single pass. With 500+ customers, consider adding virtual scrolling (e.g., react-window).

Chart rendering	Recharts renders SVG. For the weekly bar chart (7 data points) and trend chart (30 data points), performance is excellent. Beyond 100 data points, consider downsampling.
useMemo optimization	Chart data and filtered payment arrays are wrapped in useMemo with correct dependency arrays. This prevents expensive re-computations on every render.
localStorage write frequency	Every state change triggers a localStorage write (via useEffect). For frequent updates, debouncing writes could reduce I/O — not currently implemented.

11. Deployment Guide

11.1 Vercel Deployment

CablePro is optimized for deployment on Vercel, which provides automatic HTTPS, global CDN edge delivery, and seamless GitHub integration.

Step 1: Connect Repository to Vercel

44. Push the CablePro codebase to a GitHub (or GitLab/Bitbucket) repository.
45. Go to https://vercel.com → Log in → New Project.
46. Import the repository → Vercel auto-detects Next.js.
47. Configure the project settings (see below).
48. Click Deploy.

Step 2: Configure Environment Variables in Vercel

In the Vercel project dashboard → Settings → Environment Variables, add:

```
NEXT_PUBLIC_FIREBASE_API_KEY          → your Firebase API key
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN      → cablepro-xxx.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID       → cablepro-xxx
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET   → cablepro-xxx.firebasestorage.app
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID → your sender ID
NEXT_PUBLIC_FIREBASE_APP_ID           → your app ID

Apply to: Production, Preview, and Development environments.
```

Step 3: Build Settings

Setting	Value
Framework Preset	Next.js (auto-detected)
Build Command	npm run build (or next build)
Output Directory	.next (Next.js default)
Install Command	npm install
Node.js Version	20.x (LTS recommended)

Automatic Deployments

With Vercel's GitHub integration, every push to the main branch triggers an automatic production deployment. Every pull request gets its own preview URL for testing before merging.

11.2 Firebase Rules Configuration

The current CablePro implementation uses Firebase Authentication only — no Firestore or Realtime Database. No Firebase Security Rules need to be configured for the current data model (localStorage-based).

If future versions migrate data to Firestore, the following rules template should be applied:

```
// Firestore Security Rules template for future Firestore migration
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Customers: only the owning operator can read/write
    match /operators/{operatorId}/customers/{customerId} {
      allow read, write: if request.auth != null
                         && request.auth.uid == operatorId;
    }

    // Payments: only the owning operator can read/write
    match /operators/{operatorId}/payments/{paymentId} {
      allow read, write: if request.auth != null
                         && request.auth.uid == operatorId;
    }

    // Deny all other access
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

11.3 Custom Domain Setup

Adding a Domain in Vercel

49. Go to Vercel project → Settings → Domains.
50. Click 'Add' → Enter your domain (e.g., app.cablepro.in).
51. Vercel provides DNS configuration instructions (CNAME or A record).
52. Update your domain registrar's DNS settings as instructed.
53. Vercel automatically provisions an SSL certificate via Let's Encrypt.

Update Firebase Authorized Domains

After configuring the custom domain, add it to Firebase's authorized domains to prevent "auth/unauthorized-domain" errors:

54. Firebase Console → Authentication → Settings → Authorized Domains.
55. Click 'Add domain' → Enter your custom domain (e.g., app.cablepro.in).
56. Save changes.

> *The default Vercel deployment URL (xxx.vercel.app) is pre-authorized in most Firebase projects during OAuth setup. If using a custom domain, the authorization step is mandatory for Google Sign-In to work.*

12. Troubleshooting & FAQs

12.1 Common Issues & Solutions

Login: 'Invalid email or password' for seed user

The seed user uses username (not email). Enter: Username: operator | Password: operator123. Do not use the email field format.

Google Sign-In popup closes without completing

This usually indicates a browser popup blocker. Ensure popups are allowed for the CablePro domain. Alternatively, check Firebase Console → Authentication → Authorized Domains to confirm the current domain is listed.

Customer data disappears after refreshing

CablePro stores data in localStorage. If data is missing after refresh: (1) Check that localStorage is not blocked in browser settings, (2) Verify you're logged in with the same account, (3) Check if browser storage was cleared. Use Google Drive backup to recover data.

'STB Number already exists' error when adding customer

Each STB number must be unique across all customers. If a previous customer with that STB was deleted, their STB number is permanently reserved (the soft-delete anonymizes the record but does not free the STB). Contact support to force-release an STB number.

'Plan active until {date}' when recording payment

The customer's billing cycle is still active. For a 3-month plan paid in January, the system blocks new payments until April. If the operator needs to override (e.g., plan upgrade), they must first cancel the existing confirmed payment, then record the new one.

Dashboard shows ₹0 Outstanding despite overdue customers

Outstanding = 0 for Deactive customers (they don't use the service). Check the Customers list with the 'Active' filter to see only active subscribers. Overdue only applies to Active customers.

Google Drive backup fails with 'Failed to get access token'

The Google OAuth token has expired (tokens last ~1 hour). Click 'Backup Now' again — a new Google Sign-In popup will appear to refresh the token. If signed in with email/password (not Google), the Drive scope wasn't granted at login. Use 'Backup Now' which will trigger a Google auth popup.

Import CSV shows '0 customers imported'

The CSV may not match expected column headers. Required headers (case-insensitive): name (or 'customer'), phone (or 'mobile'). Optional: address, stb (or 'stb number'), package, monthly rate (or 'plan price'), status. Ensure the CSV has a header row and no BOM character.

Dark mode toggle not working in some sections

The dark mode uses Tailwind's class-based strategy (.dark class on <html>). If some elements don't respond, check that they use dark: prefixed Tailwind classes. Theme toggle calls: document.documentElement.classList.add('dark') / .remove('dark').

localStorage quota exceeded error in console

Each user's data approaches the ~5MB browser limit. Export customers and payments to CSV, then consider archiving old payment records. Future version will support cloud-first storage to eliminate this limit.

12.2 Frequently Asked Questions

Q: Can multiple operators share the same customer database?

No. The current version is a single-operator system — each Firebase account has its own isolated localStorage namespace. Multi-operator shared databases are planned for a future version using Firestore as the backend.

Q: Can CablePro work offline?

Yes — the core features (viewing customers, recording payments, browsing reports) work fully offline since all data is in localStorage. Features that require internet: Firebase login (for new sessions), Google Sign-In, Google Drive backup/restore, and new user registration.

Q: How do I migrate from my Excel spreadsheet?

Export your Excel data as CSV (File → Save As → CSV UTF-8). Ensure the CSV has headers matching CablePro's expected format (name, phone, address, stb number, package, monthly rate, status). Then use Settings → Import Customers CSV to upload the file.

Q: Can customers see their own payment history?

No. CablePro is an operator-facing tool only. There is no customer-facing portal or app. Operators can share payment receipts manually or export individual customer data to share via WhatsApp or email.

Q: Is there a limit on the number of customers?

Technically, localStorage supports ~5MB. With approximately 1KB per customer record, the practical limit is around 3,000–4,000 customers before storage issues arise. Future cloud-backed versions will remove this limitation.

Q: Can I add new payment methods like Net Banking or Cheque?

The current version only supports Cash and UPI. Adding Net Banking or other methods requires modifying the Payment interface (paymentMethod type), the New Payment form dropdown, and the payment method filter chips. This is a straightforward code change. The pie chart and reports would automatically reflect any new methods added.

Q: How accurate is the Outstanding balance calculation?

The outstanding balance is calculated in real-time based on the customer's last confirmed payment date and plan duration. A customer is marked overdue if today's date is after their coverage end date (lastPaymentDate + planDuration months). This is accurate to the day.

Q: What happens to payment records when a customer is deleted?

Payment records are preserved but anonymized. The customerName field in all payments for that customer is updated to 'Deleted Customer'. The payment amounts and dates remain intact for financial reporting purposes.

Q: Can I restore a deleted customer?

No. Customer deletion (soft delete) is not reversible through the UI. The customer's PII (name, phone, address, STB) is cleared from the record. Payment history is preserved. If you accidentally deleted a customer, you can restore the full data from a Google Drive backup made before the deletion.

Q: Does the app send SMS or WhatsApp reminders to customers automatically?

Not automatically. The Customer Detail page has a 'WhatsApp' button that opens a pre-filled WhatsApp conversation with the customer's number. Automated bulk messaging is a planned future feature.

13. Roadmap & Future Enhancements

CablePro v1.0.5 is a solid MVP covering the core operator workflow. The following enhancements are planned for future versions based on operator feedback and market requirements.

Version 1.1 — Short-term (Q1-Q2 2026)

Feature	Priority	Description
Push Notifications	High	Browser push notifications for upcoming payment due dates. Operators receive alerts 3 days before a customer's billing cycle ends.
WhatsApp Bulk Reminders	High	Batch-send WhatsApp payment reminders to all overdue customers with a single click using the WhatsApp Business API.
Payment Receipt PDF	Medium	Generate a printable/shareable PDF receipt for each individual payment, including operator logo, customer details, and payment confirmation.
Multi-STB Support	Medium	Allow a single customer to have multiple STB connections under one account (common for multi-TV households).
Area/Zone Management	Medium	Tag customers by area, zone, or sector for geographic filtering and zone-wise collection reports.
Cheque Payment Support	Low	Add Cheque as a third payment method with cheque number and bank tracking fields.

Version 2.0 — Cloud Migration (Q3 2026)

Feature	Priority	Description
Firestore Backend	Critical	Migrate from localStorage to Cloud Firestore for unlimited storage, real-time sync across devices, and data reliability.
Multi-Device Sync	Critical	Operator can use CablePro on phone and tablet simultaneously with instant data sync.
Multiple Operators / Staff	High	Add sub-operator accounts with role-based permissions (e.g., collection staff can record payments but not add/delete customers).
Customer Portal	High	A read-only web page where customers can view their payment history using their phone number and STB number as credentials.
Automated Billing	Medium	Automatically generate pending payment records on the first of each month for all active customers.
UPI Payment Integration	Medium	Generate UPI payment links or QR codes that customers can scan to pay directly into the operator's UPI account.

Analytics Dashboard V2	Medium	Monthly growth charts, churn rate tracking, revenue forecasting, and collection efficiency metrics.

Version 3.0 — Enterprise Features (2027)

Feature	Priority	Description
MSO Console	High	A parent-level dashboard for Multi-System Operators to view aggregate metrics across multiple sub-operators/franchisees.
Mobile App (React Native)	High	Native iOS and Android app for offline-first operation with background sync.
Accounting Integration	Medium	Export payment data to Tally ERP, Zoho Books, or QuickBooks for integrated bookkeeping.
Government Compliance Reports	Medium	Generate reports in formats required by TRAI (Telecom Regulatory Authority of India) for regulatory compliance.
AI-Powered Predictions	Low	Predict customer churn based on payment patterns and flag high-risk accounts for proactive retention.

14. Glossary of Terms

Term	Definition
STB (Set-Top Box)	A hardware device that decodes digital cable TV signals and outputs video to the subscriber's television. Each STB has a unique serial number used as the primary identifier in CablePro.
MSO (Multi-System Operator)	A cable TV company that operates cable infrastructure across multiple geographic areas, often managing dozens of LCOs through a franchise model.
LCO (Local Cable Operator)	An individual or small business that provides cable TV services at the last mile — directly to residential subscribers. CablePro is primarily designed for LCOs.
Billing Period	The calendar month and year for which a payment is being made (e.g., 'February 2026'). Separate from the payment date — an operator might collect February's payment in January.
Plan Duration	The number of months a single payment covers. A 3-month plan (planDuration=3) means one payment covers 3 months of service.
Overdue Customer	An active customer whose billing cycle coverage has expired. Coverage end = last payment date + planDuration months. After this date, the customer is overdue.
Soft Delete	A deletion technique where the record is not physically removed from the database but marked as deleted (isDeleted=true). The customer's PII is cleared, but payment history is preserved.
Outstanding Balance	The total amount owed by all active overdue customers. Calculated as the sum of each overdue customer's monthlyRate.
Seed User	A hard-coded demo user (username: operator) pre-loaded with sample customers and payments for demonstration and testing purposes.
Firebase UID	A unique identifier assigned by Firebase Authentication to each registered user. Used as the user's ID throughout CablePro.
hydrated	A boolean flag in the store that becomes true after localStorage data has been loaded into React state. Used to prevent SSR/client rendering mismatches.
Confirmed Payment	A payment with status='Confirmed'. Only confirmed payments reduce a customer's outstanding balance and count in financial reports.
Coverage End Date	The date after which a customer's plan expires: lastPaymentDate + planDuration months. On or after this date, the customer is considered overdue.
genId()	CablePro's internal ID generation function: Math.random().toString(36).substring(2,10). Produces 8-character random alphanumeric strings.
uKey(uid, key)	Utility function that generates a namespaced localStorage key: cp_{uid}_{key}. Ensures per-user data isolation in localStorage.

Drive.file Scope	A Google OAuth scope (https://www.googleapis.com/auth/drive.file) that grants access only to files created by the application — a least-privilege approach to Google Drive access.
TRAI	Telecom Regulatory Authority of India — the government body that regulates cable TV services in India, including subscriber reporting requirements.
UPI	Unified Payments Interface — India's real-time payment system that powers GPay, PhonePe, Paytm, and other apps. One of CablePro's two supported payment methods.
DXA	Document eXtension Arbitrary — the unit used in OOXML (docx format) where 1440 DXA = 1 inch. Used internally in Word document generation.
PWA (Progressive Web App)	A web application that can be installed on a device and offers app-like behavior including offline capability and home screen icons.

15. Appendix

Appendix A: Complete File Reference

File Path	Type	Purpose
src/app/page.tsx	Page	Login page — email/password, Google OAuth, forgot password
src/app/signup/page.tsx	Page	New operator registration form
src/app/(app)/layout.tsx	Layout	Auth guard + AppShell wrapper for all protected routes
src/app/(app)/dashboard/page.tsx	Page	Business overview with charts, KPIs, quick actions
src/app/(app)/customers/page.tsx	Page	Customer list with search, filters, and CRUD modals
src/app/(app)/customers/view/page.tsx	Page	Customer detail page wrapper (Suspense)
src/app/(app)/customers/view/CustomerDetailClient.tsx	Component	Customer detail: profile, outstanding, history
src/app/(app)/payments/page.tsx	Page	Payment list with stats, search, filters, inline actions
src/app/(app)/payments/new/page.tsx	Page	New payment form with customer search and validation
src/app/(app)/reports/page.tsx	Page	Financial analytics with charts, filters, CSV/PDF export
src/app/(app)/settings/page.tsx	Page	Profile, data summary, import/export, cloud backup, danger zone
src/app/layout.tsx	Layout	Root HTML layout: fonts, metadata, StoreProvider
src/app/globals.css	Styles	Tailwind v4 config, CSS custom properties, dark variant
src/components/AppShell.tsx	Component	Sidebar, top header, mobile bottom nav, backup reminder
src/components/Modal.tsx	Component	Reusable dialog component with ESC key and backdrop close
src/lib/store.tsx	Store	Global React Context state management with Firebase auth
src/lib/types.ts	Types	TypeScript interfaces: Customer, Payment, User
src/lib/firebase.ts	Config	Firebase SDK initialization and auth function exports
public/logo.svg	Asset	CablePro logo SVG (TV icon with brand colors)

tailwind.config.js	Config	Tailwind v4 — custom theme (see globals.css @theme)
next.config.js	Config	Next.js configuration (default for App Router)
package.json	Config	Dependencies, scripts, and project metadata

Appendix B: Seed Data Reference

The SEED_CUSTOMERS and SEED_PAYMENTS arrays in store.tsx are used to pre-populate the demo account (seed user: operator / operator123) with realistic sample data.

Seed Customers

ID	Name	Plan / Rate / Duration
c1	Rajesh Kumar	Ultra HD Gold Pack / ₹450 / 1 month
c2	Priya Sharma	Basic HD Pack / ₹300 / 1 month
c3	Amit Patel	Premium Pack / ₹500 / 1 month
c4	Sunita Devi	Ultra HD Gold Pack / ₹450 / 3 months
c5	Vikram Singh	Basic HD Pack / ₹300 / 1 month
c6	Deepak Verma	Premium Pack / ₹500 / 6 months (Deactive)
c7	Neha Gupta	Ultra HD Gold Pack / ₹450 / 12 months
c8	Rahul Saxena	Basic HD Pack / ₹300 / 1 month

Appendix C: Keyboard Shortcuts & Accessibility

Shortcut / Feature	Behavior
ESC key in Modal	Closes any open Modal component (implemented via keydown listener).
Tab navigation	All interactive elements are focusable. Modal traps focus within the dialog.
tel: links	Phone number links use the native tel: protocol, opening the device dialer on mobile.
Safe area insets	Bottom navigation uses env(safe-area-inset-bottom) for notch/home indicator clearance on iPhone.
-webkit-tap-highlight-color: transparent	Removes the default blue tap highlight on mobile WebKit for all elements (global reset in globals.css).

Appendix D: Design Tokens Quick Reference

Token Name	Value	Tailwind Class
--color-primary	#404fb5	bg-primary, text-primary, border-primary
--color-primary-light	#5c6bc0	bg-primary-light
--color-primary-dark	#303f9f	hover:bg-primary-dark
--color-teal	#0d9488	bg-teal, text-teal
--color-success	#16a34a	text-success, bg-success
--color-warning	#f59e0b	text-warning
--color-danger	#dc2626	text-danger, bg-danger
--color-orange	#ea580c	text-orange, bg-orange
--color-bg-light	#f6f6f8	bg-bg-light
--color-bg-dark	#14151e	dark:bg-bg-dark
--color-bg-card	#ffffff	bg-white
--color-bg-card-dark	#1a1f26	dark:bg-bg-card-dark
--color-text-primary	#121316	text-text-primary
--color-text-secondary	#6a6d81	text-text-secondary
--color-border	#dddee3	border-border
--color-border-dark	#2d3139	dark:border-border-dark